WEATHER AI WEB APPLICATION

Backend API & Authentication System – Project Documentation

10-PAGE PROJECT DOCUMENTATION

This document describes the backend-oriented implementation demonstrated in the supplied video. The demo shows a Node.js/Express-style backend running locally, connected to MongoDB, with API requests tested through Thunder Client.

Technology area: Web Application Development / REST API / Database Connectivity / Authentication

Demo Video

■ Click here to watch the Demo Video

Prepared for academic project documentation.

1. ABSTRACT

Project Overview

The project is a backend-supported web application that provides a structured server layer for client requests, user-related operations and application services. The supplied demonstration shows the backend running on a local server and receiving REST-style requests through Thunder Client. MongoDB is used as the database, while environment variables are used to keep configuration values outside the source code.

Purpose

Create a maintainable backend for a modern web application.

Provide REST API endpoints for client-side communication.

Connect the application to MongoDB for persistent data storage.

Support user registration/login style operations and protected application requests.

Provide a base for weather and AI-related services through external API configuration.

Expected Outcome

The completed system should start the server successfully, establish a database connection, accept valid API requests, return meaningful HTTP responses, and store or retrieve application data as required.

2. OBJECTIVES AND SCOPE

Objectives

Develop a backend server using JavaScript and Node.js.

Organize API routes into logical modules.

Use MongoDB for data persistence.

Implement authentication-related endpoints such as registration and login.

Test endpoints using Thunder Client or a similar REST client.

Use environment variables for port numbers, database URI, JWT secret and external API keys.

Create a foundation for weather information and AI-assisted features.

Scope of the Project

The project scope covers backend setup, API routing, database connectivity, authentication-related requests, configuration management and demonstration/testing. The supplied demo mainly focuses on the API layer rather than the visual frontend.

Users of the System

End users who interact with the web application.

Administrators/developers who maintain users and application data.

Frontend clients that consume the REST APIs.

3. SYSTEM ARCHITECTURE

High-Level Architecture

The application follows a client–server architecture. A frontend or API testing tool sends HTTP requests to the backend. The backend processes the request, communicates with MongoDB when required, and returns a JSON response.

Client / Thunder Client → Express/Node.js Backend → MongoDB

↓

External Weather / AI APIs

Main Components

Node.js runtime – executes the backend JavaScript application.

Express-style server – handles HTTP requests and routes.

MongoDB – stores application data.

JWT-based authentication – supports secure session/token handling.

Environment configuration – stores PORT, MongoDB URI, JWT secret and API keys.

Thunder Client – used in the demo to send and test REST requests.

Data Flow

A request is sent to an API endpoint → the route receives the request → validation/business logic is applied → MongoDB or an external service is contacted if necessary → the backend creates a response → the client receives JSON data.

4. TECHNOLOGIES AND DEVELOPMENT ENVIRONMENT

Software Technologies

JavaScript – primary programming language for the backend.

Node.js – server-side JavaScript runtime.

Express.js – commonly used framework for REST API routing.

MongoDB – NoSQL database for persistent application data.

JWT – token-based authentication mechanism.

Thunder Client – API testing tool integrated with VS Code.

Git/GitHub – suitable for source-code version control.

Configuration Management

The project uses a .env file for configuration. Typical entries include the server port, MongoDB connection URI, JWT secret, OpenWeather API key and Gemini API key. Secret values should never be committed to a public repository or included in screenshots/documentation.

Development Environment

Operating System: Windows (as shown in the supplied demo).

IDE: Visual Studio Code.

API testing: Thunder Client.

Local server: localhost with a configured backend port.

5. DATABASE AND API DESIGN

MongoDB Connectivity

MongoDB provides persistent storage for the application. The demo shows the backend reporting a successful MongoDB connection before accepting requests. The database layer can be organized with models/schemas depending on the chosen MongoDB library.

REST API Concept

The API uses HTTP methods to represent operations. POST requests are appropriate for creating or submitting data, while GET requests are commonly used to retrieve data.

Endpoints Observed in the Demonstration

POST /api/auth/register – registration-related request.

POST /api/auth/login – login-related request.

GET /api/auth/user – user retrieval/check style request.

POST /api/task – task creation/processing style request.

GET /api/signup – signup-related route visible in the API test history.

POST /api/login – login-related route visible in the API test history.

POST /api/user – user-related request visible in the API test history.

Exact request fields and response structures should be taken from the source code when preparing a final submission; the list above is based on what is visible in the supplied demo recording.

6. AUTHENTICATION AND SECURITY

Registration and Login

Authentication allows the application to distinguish registered users from unauthenticated visitors. A registration request can create a user record, while a login request verifies the supplied credentials and

can issue a token.

JWT Authentication

JSON Web Tokens can be used to represent authenticated sessions. The backend can sign a token using a secret stored in the environment configuration and validate that token on protected routes.

Client → Login API → Credentials verified → JWT issued → Client sends token with protected requests

Security Practices

Never expose database passwords or API keys in source control.

Store secrets in environment variables.

Use HTTPS in production.

Validate and sanitize incoming request data.

Hash passwords rather than storing plain-text passwords.

Use appropriate authentication middleware for protected routes.

Rotate exposed API keys immediately if they appear in screenshots or public messages.

7. DEMONSTRATION / TESTING

Demo Video

The supplied demonstration video is approximately 6 seconds long and shows the backend being tested through Thunder Client inside Visual Studio Code. The terminal indicates that the server is running and that MongoDB has connected successfully.

Demo Video

■ Click here to watch the Demo Video

Figure 1: Backend project/testing environment captured from the supplied demo video.

8. API TESTING RESULTS

Thunder Client Testing

Thunder Client is used to send requests directly to the local backend. The demonstration shows a request being prepared with an HTTP method, localhost URL and JSON body. This provides a practical way to verify that the server routes are reachable before connecting a frontend.

Figure 2: API testing view from the supplied demonstration.

Figure 3: API testing view from the supplied demonstration.

Figure 4: API testing view from the supplied demonstration.

Testing Checklist

Server starts without runtime errors.

MongoDB connection is established.

API endpoint URL is reachable.

HTTP method matches the route definition.

JSON request body uses the expected fields.

Response status and response body are checked.

9. CONCLUSION AND FUTURE ENHANCEMENTS

Conclusion

The project demonstrates the core backend workflow of a web application: starting a Node.js server, connecting to MongoDB, exposing REST-style endpoints, testing requests through Thunder Client and preparing authentication and external API configuration. The architecture can be extended into a complete Weather AI application by connecting a frontend to these services.

Future Enhancements

Develop a responsive frontend dashboard.

Add complete weather search using OpenWeather data.

Add AI-generated weather explanations or recommendations through an AI service.

Add role-based access where required.

Improve validation and centralized error handling.

Add automated unit and integration tests.

Deploy the backend and database using production-ready services.

Add logging, monitoring and rate limiting.

Reference / Demo

Demo Video

■ Click here to watch the Demo Video

Note: This documentation is prepared from the supplied demo recording and the project configuration visible in the provided screenshots. Replace the placeholder project/student details with your college-required information before submission.

DEMO VIDEO – CLICK TO WATCH

■ OPEN DEMO VIDEO

The demo link is embedded as a clickable hyperlink in this PDF. Tap the link while viewing the PDF to open the video.